

Asynchronous JavaScript

Callbacks · Callback Hell · Promises · Async/Await

A Practical Reference Guide

Table of Contents

1. Synchronous vs Asynchronous Programming
2. Callback Functions
3. Callback Hell
4. Promises
5. The 3 States of a Promise
6. Promise Methods
7. Async / Await
8. Error Handling in Async Code
9. The Event Loop & Microtask Queue
10. Summary Comparison

1. Synchronous vs Asynchronous Programming

Theory

Synchronous programming means code executes line by line, in order. Each statement waits for the previous one to finish before running. This is simple to reason about but can block the program if a task takes a long time (e.g. reading a large file or waiting for a network response).

Asynchronous programming allows a program to start a long-running task (like a network request or a timer) and continue executing other code without waiting for it to finish. When the task completes, a callback, promise, or async/await mechanism handles the result.

JavaScript is **single-threaded** — it can only do one thing at a time. Asynchronous behavior is made possible by the browser/Node.js runtime (Web APIs, timers, the event loop) working alongside the JavaScript engine.

Key idea: Synchronous = blocking, executes immediately in order. Asynchronous = non-blocking, execution can be deferred until a task completes.

Example: Synchronous Code

```
console.log("Step 1");
console.log("Step 2");
console.log("Step 3");

// Output (always in this order):
// Step 1
// Step 2
// Step 3
```

Example: Asynchronous Code

```
console.log("Step 1");

setTimeout(() => {
  console.log("Step 2 (delayed)");
}, 2000);

console.log("Step 3");

// Output:
// Step 1
// Step 3
// Step 2 (delayed)  -- printed after 2 seconds
```

Notice that `"Step 3"` runs before `"Step 2"`, even though it appears later in the code. This is because `setTimeout` is asynchronous — it schedules the callback to run later, without blocking the rest of the script.

2. Callback Functions

Theory

A **callback function** is a function passed as an argument to another function, to be executed after some operation completes. Callbacks were the original way JavaScript handled asynchronous operations before Promises existed.

Simple Callback Example

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}

greet("Shrawani", function() {
  console.log("Callback executed!");
});
```

Asynchronous Callback Example

```
function fetchData(callback) {
  setTimeout(() => {
    const data = { id: 1, name: "Laptop" };
    callback(data);
  }, 1500);
}

fetchData((result) => {
  console.log("Data received:", result);
});
```

Error-First Callback Pattern

A common Node.js convention where the first argument of a callback is reserved for an error (if any), and the second argument holds the result.

```
function readFile(path, callback) {
  if (!path) {
    return callback(new Error("Path is required"), null);
  }
  callback(null, "File content here");
}

readFile("data.txt", (err, data) => {
  if (err) {
    console.error("Error:", err.message);
  } else {
    console.log("Success:", data);
  }
});
```

3. Callback Hell

Theory

Callback hell (also called the "Pyramid of Doom") occurs when multiple asynchronous operations depend on each other and are nested inside one another's callbacks. This produces code that grows horizontally, becoming difficult to read, debug, and maintain.

Example

```
getUser(userId, (err, user) => {
  if (err) throw err;
  getOrders(user.id, (err, orders) => {
    if (err) throw err;
    getOrderDetails(orders[0].id, (err, details) => {
      if (err) throw err;
      getPaymentInfo(details.paymentId, (err, payment) => {
        if (err) throw err;
        console.log("Payment info:", payment);
        // ...and it keeps nesting further
      });
    });
  });
});
```

Problems with callback hell:

- Code becomes deeply nested and hard to read ("pyramid" shape)
- Error handling has to be repeated at every level
- Difficult to reason about execution order and debug
- Hard to reuse or test individual steps
- Adding/removing a step means re-indenting a large block of code

The Solution

Callback hell is solved using **Promises** (which allow chaining with `.then()` instead of nesting) and later, **async/await** (which allows asynchronous code to be written in a flat, synchronous-looking style).

4. Promises

Theory

A **Promise** is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value. Promises were introduced in ES6 to provide a cleaner alternative to nested callbacks.

A promise acts as a placeholder for a value that is not yet known when the promise is created, but will be resolved at some point in the future.

Creating a Promise

```
const myPromise = new Promise((resolve, reject) => {
  const success = true;

  setTimeout(() => {
    if (success) {
      resolve("Operation succeeded!");
    } else {
      reject("Operation failed!");
    }
  }, 1000);
});
```

Consuming a Promise

```
myPromise
  .then((result) => {
    console.log("Success:", result);
  })
  .catch((error) => {
    console.error("Error:", error);
  })
  .finally(() => {
    console.log("Promise settled (done, either way)");
  });
```

Rewriting Callback Hell with Promises

```
getUser(userId)
  .then((user) => getOrders(user.id))
  .then((orders) => getOrderDetails(orders[0].id))
  .then((details) => getPaymentInfo(details.paymentId))
  .then((payment) => console.log("Payment info:", payment))
  .catch((err) => console.error("Error:", err));
```

Each `.then()` returns a new promise, allowing operations to be **chained** in a flat, readable sequence instead of nested one inside another. A single `.catch()` at the end can handle errors from anywhere in the chain.

5. The 3 States of a Promise

Theory

A promise is always in one of three mutually exclusive states:

State	Meaning
PENDING	Initial state. The asynchronous operation has not completed yet.
FULFILLED	The operation completed successfully. The promise now has a resulting value (<code>resolve()</code> was called).
REJECTED	The operation failed. The promise now has a reason for failure (<code>reject()</code> was called).

Important: Once a promise is fulfilled or rejected, it is considered **settled** — its state and value are locked and cannot change again.

State Transition Diagram (conceptual)



Example Demonstrating States

```
const p1 = new Promise((resolve) => {
  setTimeout(() => resolve("Done!"), 1000);
});
// p1 is PENDING immediately after creation

console.log(p1); // Promise { <pending> }

setTimeout(() => {
  console.log(p1); // Promise { "Done!" } -> now FULFILLED
}, 1500);
```

6. Promise Methods

Theory

The `Promise` object provides several static methods for working with one or more promises together.

Promise.all()

Waits for **all** promises to fulfill. If **any one** promise rejects, the entire `Promise.all()` immediately rejects with that reason.

```
const p1 = Promise.resolve(10);
const p2 = Promise.resolve(20);
const p3 = Promise.resolve(30);

Promise.all([p1, p2, p3])
  .then((results) => console.log(results)) // [10, 20, 30]
  .catch((err) => console.error(err));
```

Promise.allSettled()

Waits for **all** promises to settle (either fulfilled or rejected), and never short-circuits. Returns an array of result objects describing the outcome of each promise.

```
const p1 = Promise.resolve("Success");
const p2 = Promise.reject("Failed");

Promise.allSettled([p1, p2]).then((results) => {
  console.log(results);
  // [
  //   { status: "fulfilled", value: "Success" },
  //   { status: "rejected", reason: "Failed" }
  // ]
});
```

Promise.race()

Settles as soon as **any one** promise settles (fulfilled or rejected) — whichever happens first "wins".

```
const fast = new Promise((resolve) => setTimeout(resolve, 100, "Fast"));
const slow = new Promise((resolve) => setTimeout(resolve, 500, "Slow"));

Promise.race([fast, slow]).then((result) => {
  console.log(result); // "Fast"
});
```

Promise.any()

Settles as soon as **any one** promise fulfills. If **all** promises reject, it rejects with an `AggregateError`.

```
const p1 = Promise.reject("Error 1");
const p2 = new Promise((resolve) => setTimeout(resolve, 200, "Success"));

Promise.any([p1, p2]).then((result) => {
  console.log(result); // "Success"
});
```

Quick Comparison

Method	Resolves When	Rejects When
<code>Promise.all()</code>	All promises fulfill	Any one promise rejects
<code>Promise.allSettled()</code>	All promises settle	Never rejects
<code>Promise.race()</code>	First promise to settle (fulfilled)	First promise to settle (rejected)
<code>Promise.any()</code>	First promise to fulfill	All promises reject

Promise.resolve() / Promise.reject()

Utility methods to create an already-settled promise directly.

```
const resolved = Promise.resolve("Already done");  
const rejected = Promise.reject("Already failed");
```

7. Async / Await

Theory

`async` / `await`, introduced in ES2017, is **syntactic sugar built on top of Promises**. It allows asynchronous code to be written in a flat, synchronous-looking style, making it much easier to read and reason about than chained `.then()` calls.

- The `async` keyword before a function makes it always return a Promise.
- The `await` keyword pauses execution of the async function until the awaited promise settles, then returns its resolved value (or throws if rejected).
- `await` can only be used inside an `async` function (or at the top level of a module).

Basic Example

```
function delay(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function run() {
  console.log("Start");
  await delay(2000);
  console.log("2 seconds later");
}

run();
```

Rewriting the Promise Chain with Async/Await

```
async function getPaymentInfo(userId) {
  const user = await getUser(userId);
  const orders = await getOrders(user.id);
  const details = await getOrderDetails(orders[0].id);
  const payment = await getPaymentInfo(details.paymentId);
  console.log("Payment info:", payment);
  return payment;
}
```

This reads almost exactly like synchronous code, yet each `await` is non-blocking under the hood — it pauses only the async function, not the entire program.

Running Promises in Parallel with Async/Await

Awaiting promises one after another (sequentially) is slower than necessary when the operations don't depend on each other. Use `Promise.all()` with `await` to run them in parallel.

```
// Sequential (slower - total time = sum of all delays)
const a = await fetchA();
const b = await fetchB();

// Parallel (faster - total time = longest single delay)
const [a, b] = await Promise.all([fetchA(), fetchB()]);
```

8. Error Handling in Async Code

Theory

Since `async` functions return promises, errors thrown inside them automatically reject the returned promise. The standard way to handle these errors is with a `try...catch` block.

Try/Catch with Async/Await

```
async function fetchData(id) {
  try {
    const user = await getUser(id);
    const orders = await getOrders(user.id);
    return orders;
  } catch (error) {
    console.error("Something went wrong:", error.message);
  } finally {
    console.log("Request attempt finished");
  }
}
```

Comparison: With Promises, errors are handled using `.catch()` at the end of a chain. With async/await, the same is achieved using a `try...catch` block — both approaches ultimately rely on the same underlying Promise rejection mechanism.

9. The Event Loop & Microtask Queue

Theory

JavaScript is single-threaded, so it relies on the **event loop** to handle asynchronous operations without blocking. The event loop constantly checks whether the **call stack** is empty, and if so, moves pending tasks from queues onto the stack for execution.

There are two important queues:

- **Macrotask queue (Callback queue):** Holds tasks like `setTimeout`, `setInterval`, and I/O callbacks.
- **Microtask queue:** Holds tasks like resolved Promise callbacks (`.then`, `.catch`, `.finally`) and `async/await` continuations.

Priority rule: After each task from the call stack finishes, the event loop empties the *entire* microtask queue before moving on to the next macrotask. This means Promise callbacks always run before `setTimeout` callbacks, even if the timeout delay is 0.

Example: Execution Order

```
console.log("1: Sync start");

setTimeout(() => console.log("2: Macrotask (setTimeout)"), 0);

Promise.resolve().then(() => console.log("3: Microtask (Promise)"));

console.log("4: Sync end");

// Output order:
// 1: Sync start
// 4: Sync end
// 3: Microtask (Promise)
// 2: Macrotask (setTimeout)
```

10. Summary Comparison

Concept	Description
Callback	Function passed into another function, called after a task completes
Callback Hell	Deeply nested callbacks that are hard to read/maintain
Synchronous	Code runs line by line, blocking until each step finishes
Asynchronous	Code can run without waiting, using callbacks/promises/async-await
Promise	Object representing a future value of an async operation
Pending / Fulfilled / Rejected	The three states a promise can be in
.then() / .catch() / .finally()	Handle success, failure, and cleanup of a promise
Promise.all / allSettled / race / any	Combine and coordinate multiple promises
async/await	Syntax that makes promise-based code look synchronous
Event loop	Mechanism that manages execution order of sync and async code

Conclusion: JavaScript evolved from callback-based asynchronous handling, which led to unmanageable "callback hell," to Promises which allow clean chaining and centralized error handling, and finally to async/await which lets asynchronous logic be written in a readable, synchronous style — all built on the same underlying event loop and microtask queue mechanics.